PROJECT EXPLAINER

Building One Shared Architecture Foundation for Six Engineering Teams

Eliminating duplicated infrastructure work across application, platform, DevOps, security, data and AI teams

A plain-English explanation of the project, every component,

the end-to-end workflow, and the architecture behind it.

Who this document is for

It assumes no prior architecture knowledge. Every technical term is explained the first time it appears, and there is a full glossary at the end.

It can be read straight through, or used as a reference: Section 4 explains the parts, Section 5 explains how they fit together, and Section 6 explains what actually happens day to day.

Contents

1. Executive Summary

1.1 The project in one paragraph

Six different engineering teams were each building the same underlying plumbing for their own services. Every team had invented its own way to design a service, expose an API, deploy code, produce logs, and handle failures. The work was almost identical, but because it was done six separate times it was inconsistent, expensive to maintain, and hard to support. This project replaced that with a single shared foundation: a small set of proven, written-down building blocks that every team uses instead of inventing their own. Crucially, the foundation was not just published and forgotten. It was turned into working templates and libraries, wired into automated checks, discussed in architecture reviews, and taught to engineers through mentorship, so that it was genuinely adopted rather than politely ignored.

1.2 The everyday analogy

Imagine a large housing developer with six construction crews. Each crew designs its own electrical wiring, its own plumbing layout, its own door sizes and its own fire alarms. Every house works, roughly. But no two houses are the same, no electrician can be moved between crews without retraining, no spare part fits more than one house, and when a fault is found in one design nobody knows whether the other five have it too.

This project was the equivalent of that developer publishing one set of approved building plans, one standard door size, one wiring diagram and one safety code. Crews still build different houses for different customers, but the parts underneath are the same. Now a fault gets fixed once, an inspector knows exactly what to look for, and a new crew member is productive on day one.

1.3 What the original description actually means, phrase by phrase

The project is often described in a single dense sentence. Here is what each part of it means in ordinary language.

Phrase used	What it means in plain English
“Eliminated duplicated infrastructure work”	Several teams were separately building the same behind-the-scenes machinery. That repeated effort was removed.
“across application, platform, DevOps, security, data and AI teams”	The change was not confined to one team. It covered every engineering group in the organisation, each with different needs.
“defining reusable architectural patterns”	Writing down a small menu of approved, proven ways to build a service, so nobody has to invent a new shape from scratch.
“API standards”	One agreed way for services to talk to each other: how requests are named, how errors are reported, how versions change.
“deployment practices”	One agreed, repeatable route for getting code from a developer's laptop into production safely.

Phrase used	What it means in plain English
“observability conventions”	One agreed way for every service to report what it is doing, so problems can be found quickly.
“reliability guardrails”	Safe defaults, already switched on, that stop a small failure spreading into a full outage.
“driving adoption through architecture reviews and mentorship”	Making sure the standards were actually used: designs were reviewed before being built, and engineers were coached rather than policed.

1.4 What changed, in practical terms

Before	After
Each team invented its own service structure.	A short catalogue of approved patterns to choose from.
Six different API styles, so integration meant guesswork.	One contract style, so any team can call any service.
Six deployment pipelines, each maintained by one person.	One shared pipeline template, maintained centrally.
Logs in six formats. Debugging meant learning each one.	One log and metric format, so one dashboard fits all.
Timeouts and retries added ad hoc, usually after an outage.	Safe defaults built in from the first line of code.
Design mistakes discovered in production.	Design mistakes discovered in a 30-minute review.
Knowledge locked in individuals.	Knowledge written down and actively taught.

The core idea in one line

Solve the hard, repetitive problems once. Give the answer to everyone as something they can install rather than something they must read. Then keep improving it using what real teams hit in production.

2. The Problem, in Plain English

2.1 Why six teams built the same thing six times

Nobody set out to duplicate work. It happened because each team had a real deadline and no obvious shared alternative. When an engineer on the data team needed a service to retry a failed call, the fastest route was to write ten lines of retry code themselves. Writing those ten lines took an afternoon. Finding out whether another team had already solved it, getting access to their code, and adapting it would have taken a week. Every individual decision was rational. The sum of them was expensive.

Three forces kept the duplication going:

- There was no shared default. If nothing exists to reuse, building your own is not laziness, it is the only option.
- There was no shared vocabulary. Two teams could have solved the same problem and never realised it, because one called it “throttling” and the other called it “rate control”.
- There was no moment where anyone looked across teams. Designs were reviewed inside a team, by people who only saw that team's work, so cross-team repetition was invisible.

2.2 The five real costs of duplication

Cost	What it looked like in practice
Wasted engineering time	The same problem, solved from scratch, up to six times. Every one of those hours could have gone into features customers actually asked for.
Inconsistent behaviour	One service retried failed calls three times, another retried forever. Under load the second one turned a small blip into an outage by hammering an already-struggling dependency.
Slow, painful debugging	During an incident, engineers had to read six different log formats to trace a single customer request across services. Time spent translating was time not spent fixing.
Security and compliance gaps	A control applied properly in one team's pipeline was missing in another's, and no one could easily tell which was which. Proving compliance meant six separate investigations.
Knowledge that could not move	An engineer moving from the platform team to the AI team started again from zero, because nothing they knew about how to build and ship a service still applied.

Figure 1 | The problem this project solved

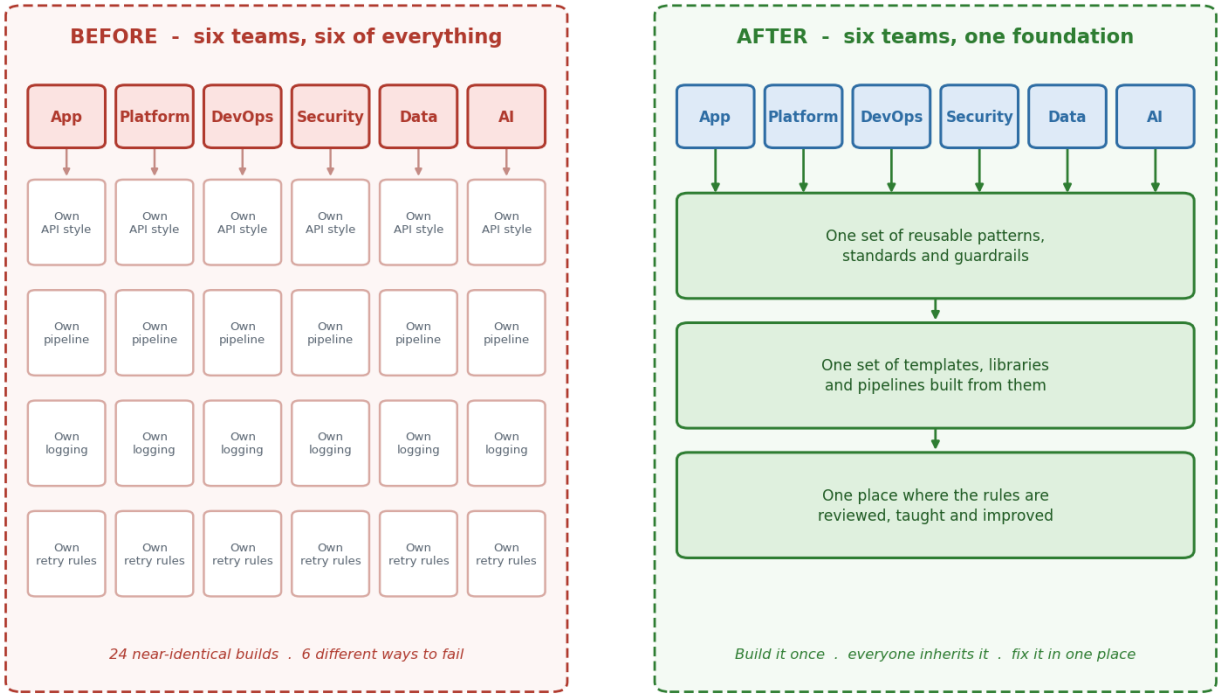

Figure 1: the same four problems solved separately by six teams, versus solved once and inherited by all six.

2.3 Why this is an architecture problem, not a tooling problem

It is tempting to think the fix is simply “buy a better tool”. It is not. A tool with no agreed way of using it just becomes a sixth way of doing things. The missing pieces were decisions, not products: which shape should a service be, what should an error look like, when is a deployment considered safe. Those are architectural questions. Tools then implement the answers. That ordering matters, and getting it backwards is the most common reason platform projects fail.

3. Goals and Scope

3.1 What the project set out to achieve

Goal	How it was judged
Remove repeated infrastructure work	New services stop hand-building logging, deployment, retries and monitoring. They inherit them.
Make services behave consistently	Two services written by two different teams fail, log and recover in the same recognisable way.
Make the right way the easy way	Following the standard should be faster than ignoring it. If it is slower, the standard is wrong.
Make good design cheap to get	A team should be able to get useful architectural help in days, not queue for weeks.
Keep the standards honest	The standards must change when reality disagrees with them, and there must be a route to do things differently with good reason.

3.2 What the project deliberately did not do

Scope discipline mattered as much as ambition. The following were explicit non-goals.

- **It did not standardise business logic.** Teams remained completely free to decide what their service does. Only the plumbing underneath was standardised.
- **It did not force rewrites.** Existing services were not required to be rebuilt. New work used the standards; old work migrated only where there was a clear benefit.
- **It did not become a gatekeeper.** The review process was designed to be fast and advisory in most cases. A process that blocks teams gets routed around.
- **It did not aim for one hundred percent compliance.** A documented, time-boxed exception is a healthy outcome. Pretending every case fits the standard is not.

3.3 The six teams and what each one needed

A standard that only suits one kind of team will be rejected by the other five. Each group had genuinely different pressures, and the standards had to accommodate all of them.

Team	What they build	What they most needed from the foundation
Application	Customer-facing services and user journeys	Speed. A way to launch a new service in hours without thinking about plumbing.
Platform	The shared runtime other teams deploy onto	Predictability. Fewer unique snowflake services to support.

Team	What they build	What they most needed from the foundation
DevOps / SRE	Pipelines, environments, production operations	Uniformity. One pipeline and one dashboard shape rather than six.
Security	Controls, scanning, access, compliance evidence	Provability. Controls applied automatically and demonstrable on demand.
Data	Pipelines, warehouses, streaming and data services	Contracts. Stable, versioned interfaces so upstream changes do not silently break pipelines.
AI / ML	Models, training jobs, inference endpoints	Guardrails. Inference is slow and expensive, so timeouts, limits and fallbacks matter more here than anywhere.

4. The Components, Explained End to End

The foundation is made of nine components. Five of them are the standards themselves, the written decisions that say how things are done. Two are the human mechanisms that get those decisions adopted. Two are the supporting machinery that keeps the whole thing honest. Each one is described below using the same structure, so they can be compared directly.

#	Component	In one line
1	Reusable architectural patterns	A short menu of approved ways to shape a service.
2	API standards	One agreed way for services to talk to each other.
3	Deployment practices	One safe, repeatable route into production.
4	Observability conventions	One agreed way for services to report what they are doing.
5	Reliability guardrails	Safe defaults that stop small failures becoming outages.
6	Architecture reviews	A short conversation that catches problems before they are built.
7	Mentorship and enablement	Teaching the standards so people follow them willingly.
8	Reusable assets (the paved road)	The standards turned into templates, libraries and pipelines.
9	Decision records and waivers	A written trail of what was decided, and permission to differ.

> **How to read each component below**
>
> Every component is described with the same six fields: what it is, the problem it solves, what is actually inside it, how a team uses it day to day, what goes wrong without it, and how you can tell it is working.

4.1 Component 1 — Reusable Architectural Patterns

What it is. A small, deliberately short catalogue of approved ways to build a service. Each entry describes a shape that has already been proven to work here, along with when to use it and when not to. A pattern is not code. It is a recipe: the arrangement of parts, and the reasoning behind the arrangement.

The problem it solves. Without a catalogue, every engineer designs from a blank page. Two engineers solving the same problem in different teams will produce two different structures, both defensible, neither compatible. The catalogue removes that blank page. Choosing becomes a five-minute decision instead of a two-week design exercise.

What is actually in it

Each catalogue entry is one page and contains the following fields. Consistency matters here: if entries have different structures, people stop reading them.

Field in the entry	What it contains and why
Name	A short, memorable label. Shared vocabulary is half the value. Once everyone says “request-response with cache-aside”, they are talking about the same thing.
Problem it solves	The situation this pattern is for, written as a scenario rather than as theory.
When to use it	Concrete triggers. For example: the caller needs an answer immediately and can wait up to 300 milliseconds.
When NOT to use it	The most valuable field. Patterns cause damage when applied to the wrong problem, and this field is what prevents that.
The shape	A simple diagram of the parts and how requests flow between them.
Trade-offs	What you give up by choosing it. Every pattern costs something, and hiding the cost destroys trust in the catalogue.
Reference implementation	A link to working code that already implements the pattern, so it can be copied rather than interpreted.
Owner and last reviewed date	A pattern with no owner and no recent review date is a rumour, not a standard.

The patterns themselves

Pattern	Plain-English description
Request / response service	The caller asks a question and waits for the answer. The default choice for anything a user is actively waiting on, such as loading a page.
Event-driven / asynchronous	The caller announces that something happened and moves on. Other services react in their own time. Used when the work is slow or when several unrelated things must happen afterwards.

Pattern	Plain-English description
Batch / scheduled job	Work that runs on a timer over a large volume of records. Nightly reports and bulk recalculations. Nobody is waiting, so throughput matters more than speed.
Cache-aside read	Check a fast temporary store first; only go to the slow source if the answer is not there. Used for data read far more often than it changes.
Gateway-fronted service	A single front door handles authentication, rate limiting and routing, so individual services do not each re-implement those concerns.
Streaming pipeline	A continuous flow of records processed as they arrive rather than in nightly batches. The default for data and telemetry work.
Model inference endpoint	A service that wraps a machine-learning model. Treated as its own pattern because it is slow, expensive per call, and needs a fallback answer when it cannot respond in time.

How a team uses it day to day. At the start of a piece of work, an engineer opens the catalogue and matches their need to an entry. They copy the reference implementation, and they record which pattern they chose in their design brief. If nothing in the catalogue fits, that is treated as useful signal rather than failure: it either means a new pattern is needed, or that the problem has been misunderstood, and both are worth ten minutes of conversation.

What it looks like when this is missing. Long design debates that restart from first principles every time. Services that are subtly different from each other for no deliberate reason. New joiners who cannot predict how any given service is put together.

How you know it is working. Design conversations get shorter and more specific. People start referring to patterns by name in ordinary conversation. The number of genuinely novel service shapes per quarter drops close to zero.

4.2 Component 2 — API Standards

What it is. An API is the doorway a service exposes so other services can use it. API standards are the agreed rules for what that doorway looks like: how it is named, how it reports success and failure, how it changes over time, and how it proves who is calling.

The problem it solves. When every team designs its own doorway, integrating two services becomes an archaeology exercise. One service returns a 200 status with an error hidden inside the body, another returns a 400 with a plain string. Callers must write custom handling for every service they talk to, and that handling is where bugs live.

What is actually in it

Rule area	What the standard says, in plain terms
Naming	Addresses describe things, not actions, and are consistently plural and lowercase. A caller can guess an address correctly without reading documentation.
Request and response shape	One agreed envelope. Data always sits in the same place, and lists always report their total count the same way, so client code can be written once and reused.
Errors	Every failure returns a machine-readable code, a human-readable message, and an identifier that can be searched for in the logs. No silent failures and no error text that only makes sense to the author.
Status codes	Agreed meanings. “You sent something wrong” and “we broke” must be distinguishable, because callers retry one and not the other.
Versioning	Changes that break existing callers require a new version. The old version keeps running for a published period. This is the rule that lets teams move independently.
Pagination	One agreed way to ask for large result sets in pieces, so no caller can accidentally request ten million records and take the service down.
Authentication and authorisation	One agreed way to prove identity and one agreed way to express permissions. Individual services never invent their own login mechanism.
Machine-readable contract	Every API publishes a specification file. This is what makes automated checking possible, and it is what turns the standard from advice into something enforceable.
Idempotency	Repeating the same write request must not repeat its effect. Networks retry, so without this rule a customer eventually gets charged twice.

How a team uses it day to day. The API specification is written before the code. A linter checks it against the standard automatically in the pipeline, so a naming or error-format mistake is caught in seconds rather than in a review weeks later. Client libraries are generated from the specification instead of hand-written.

What it looks like when this is missing. Integration work takes far longer than estimated. Every new caller discovers a fresh surprise. A change made innocently by one team silently breaks another team's service in production, because there was no agreed definition of what counts as a breaking change.

How you know it is working. A team can integrate with a service they have never used before, without asking its owners any questions. The number of production incidents caused by an unannounced interface change falls to near zero.

4.3 Component 3 — Deployment Practices

What it is. The single agreed route that code takes from a developer's machine to production, including every check it must pass and every safety step it must go through on the way.

The problem it solves. When each team builds its own route, quality depends on who built it. One pipeline scans for vulnerable dependencies, another does not. One can roll back in a minute, another needs a person to remember a manual sequence at two in the morning. Deployment becomes the riskiest moment in the week rather than a routine one.

The stages, in order

Stage	What happens and why it matters
1. Build	Source code is turned into one immutable package. The exact same package is used in every environment. Rebuilding for production is forbidden, because it reintroduces the risk that what was tested is not what ships.
2. Test	Automated tests run. Unit tests check individual pieces; contract tests check that the service still honours the interface its callers depend on.
3. Scan	Dependencies, container images and configuration are checked for known vulnerabilities and for secrets accidentally committed to the repository.
4. Sign	The package is cryptographically signed and recorded, so production can later verify that it is running exactly what the pipeline produced and nothing else.
5. Policy check	Automated rules run: resource limits present, health checks defined, no hard-coded credentials, an owner recorded. Failing a rule fails the build.
6. Deploy to staging	Released to a production-like environment first. Smoke tests confirm the service starts, answers, and can reach its dependencies.
7. Progressive release	Sent to a small slice of real traffic, often one to five percent. Error rate and latency are compared against the current version automatically.
8. Automatic rollback	If the comparison looks worse, traffic returns to the previous version without a human deciding. This is the step that converts a potential outage into a non-event.
9. Record	What was deployed, by whom, when, and from which change is written to an audit trail. Security and compliance read this instead of interviewing engineers.

How a team uses it day to day. A team does not build any of this. They inherit a pipeline template. Merging a change starts the sequence automatically. Their only routine responsibility is keeping their tests meaningful.

What it looks like when this is missing. Deployments are scheduled for late at night because they are frightening. Rollback is a document rather than a button. Nobody can answer “what exactly is running in production right now” with confidence.

How you know it is working. Deploys happen during business hours without ceremony, and their frequency rises while the proportion that cause an incident falls. Time to recover from a bad release is measured in minutes.

4.4 Component 4 — Observability Conventions

What it is. Observability means being able to tell what a system is doing from the outside. The conventions are the agreed rules for what every service must report and in exactly what format, so that all services can be understood using the same tools and the same habits.

The problem it solves. Modern requests cross many services. If each writes its own style of log, following one customer's request through the system means manually stitching together six unrelated records. During an incident that translation work is the difference between a fifteen-minute recovery and a three-hour one.

The three signals, explained simply

Signal	What it is and what it answers
Logs	A dated line of text recording that something specific happened. Logs answer “what exactly happened to this one request?” They are the detail.
Metrics	Numbers counted over time, such as requests per second or error percentage. Metrics answer “is the system healthy right now, and is that unusual?” They are the summary.
Traces	A single request's complete journey across every service it touched, with the time spent in each. Traces answer “where did the time actually go?” They are the map.

What the conventions require

Convention	What it means and why
Structured logs	Logs are written as machine-readable fields rather than free-form sentences, so they can be searched and filtered rather than read line by line.
Mandatory fields	Every log line carries the same core fields: timestamp, service name, environment, severity, and the correlation identifier.
Correlation identifier	One identifier is generated when a request first enters the system and passed to every service it touches. This single rule is what makes end-to-end tracing possible at all.
Consistent metric names	The same measurement is named identically everywhere, so one dashboard definition works for every service without editing.
Standard labels	Every metric is tagged with service, environment, version and region, so results can be sliced consistently.
No sensitive data	Personal data, credentials and tokens are never written to logs. This is enforced automatically, because relying on memory guarantees eventual failure.
Health and readiness signals	Every service reports whether it is alive and whether it is ready for traffic, in the same way, so the platform can manage it without special cases.
Dashboards and alerts as code	Monitoring is defined in files stored alongside the service, so it is created automatically and cannot drift out of date.

How a team uses it day to day. Mostly, they do not have to think about it. The shared library emits the correct format automatically and propagates the correlation identifier without being asked. A team gets a working dashboard on the day their service is created, before it has any traffic.

What it looks like when this is missing. Incidents begin with twenty minutes of “which service is this even coming from?”. Alerts fire for things nobody acts on, so people stop reading them. Root cause is guessed rather than found.

How you know it is working. Any engineer can trace a single request across service boundaries without help. Time-to-detect and time-to-diagnose both fall measurably.

4.5 Component 5 — Reliability Guardrails

What it is. A set of protective defaults, switched on from the beginning, that limit how badly things can go wrong when a dependency slows down or fails. They are called guardrails because they do not steer, they simply prevent the worst outcome.

The problem it solves. Distributed systems fail in a specific and predictable way. One slow service causes its callers to pile up waiting, which makes them slow, which makes their callers pile up. A problem in one unimportant corner becomes a full outage in minutes. Guardrails break that chain.

The guardrails, one by one

Guardrail	What it does, in plain terms
Timeout	A maximum time to wait for an answer before giving up. Without it, a caller waits forever and holds resources it cannot release. This is the single most valuable guardrail.
Retry with backoff and jitter	If a call fails, try again, but wait longer between each attempt and add a small random delay. The randomness stops every client retrying in perfect unison and finishing off a service that was about to recover.
Retry budget	A cap on how much of total traffic may be retries. Retries are helpful in small doses and are an attack on your own system in large ones.
Circuit breaker	After repeated failures, stop calling the struggling service entirely for a short period and fail fast instead. This gives the dependency room to recover rather than being hammered while it is down.
Bulkhead	Separate pools of resources per dependency, so one slow dependency cannot consume every connection and starve everything else.
Rate limit	A ceiling on requests accepted per caller per period. Protects a service from being overwhelmed, whether by an attack or by a well-meaning script.
Graceful degradation	A defined, reduced answer when the ideal one is unavailable: a cached value, a default, or a clearly marked partial result. Better than an error page.
Health checks	A standard way to report liveness and readiness, so the platform can restart or withhold traffic from an unhealthy instance automatically.
Resource limits	Every service declares the memory and processing power it may use, so one runaway service cannot starve its neighbours.
SLOs and error budgets	A written target for how reliable a service should be, plus an agreed allowance for falling short. When the allowance is used up, the team pauses new features and fixes reliability. It turns a vague argument into a number.

> **Why these are defaults, not options**
>
> Guardrails are the classic example of work nobody does under deadline pressure, because their benefit only appears on a bad day. Making them the default, already switched on in the template, means a team must make a deliberate decision to remove protection rather than a deliberate decision to add it. That single inversion is most of the value of this component.

How a team uses it day to day. The values arrive pre-set in the service template and the shared library. A team tunes the numbers for their own traffic, and records the SLO their service is held to. Removing a guardrail requires a written reason.

What it looks like when this is missing. Outages that spread. Post-incident reviews that conclude “we should add a timeout”, repeatedly, for years.

How you know it is working. Incidents stay contained within one service instead of cascading. The number of separate services affected by an average incident falls.

4.6 Component 6 — Architecture Reviews

What it is. A short, structured conversation about a design before it is built. It is the main point where cross-team duplication is spotted, because it is the only place where someone sees all six teams' work.

The problem it solves. Design flaws are cheap to fix on paper and expensive to fix in production. More importantly, nobody inside a single team can see that the thing they are about to build already exists elsewhere. The review supplies that missing view.

How a review is run

Element	How it works and why
Trigger	A review is required for anything new, anything crossing a team boundary, anything handling sensitive data, and anything that would depart from a standard. Routine changes inside an existing pattern do not need one.
Pre-read	A one-page brief is circulated in advance. Reading it during the meeting wastes everyone's time, so the meeting starts with everyone already informed.
Attendance	The proposing engineer, an architect, and a representative from whichever specialisms are relevant, usually security and operations. Deliberately small.
Duration	Thirty minutes by default. A long review usually means the brief was unclear, not that the design is complex.
The four questions	Does this already exist? Does it follow the standards? How will it fail? Who will operate it? Almost every valuable finding comes from one of these four.
Tone	Advisory by default. The purpose is to improve the design and to surface reuse, not to grant permission. A review that feels like an exam gets avoided.
Outcome	One of three: proceed, proceed with named conditions, or a time-boxed waiver to do something differently. Always written down, never left implied.
Follow-up	Conditions are tracked as real work items. An unfollowed condition means the review achieved nothing.

The one-page design brief

The brief is deliberately short. Its job is to force clarity, not to produce a document.

- What this does, in two sentences a non-engineer could understand.
- Who calls it, and what they expect back.
- Which pattern from the catalogue it uses, and why that one.
- What data it touches, and whether any of it is sensitive.
- How it fails: what happens when each dependency is slow or unavailable.
- Expected load today, and expected load in a year.
- Who will operate it at three in the morning.

What it looks like when this is missing. Duplicated services discovered months later. Security consulted after launch, when changes are expensive. Designs that work in testing and fall over under real traffic.

How you know it is working. Reviews start finding fewer problems over time, because teams have internalised the questions and pre-empt them. Teams begin requesting reviews voluntarily because they find them useful.

4.7 Component 7 — Mentorship and Enablement

What it is. The deliberate teaching of the standards to engineers, through pairing, office hours, worked examples and written explanations of reasoning. It is the difference between a rule people obey and a rule people understand.

The problem it solves. A standard that is only enforced is followed to the letter and defeated in spirit. Engineers who do not understand why a rule exists will satisfy the automated check and still produce something fragile. Mentorship converts compliance into judgement, which is the only thing that generalises to situations the standard did not anticipate.

What it consists of in practice

Activity	What it involves
Office hours	A regular open slot where any engineer can bring a design problem with no preparation and no formal process. Removes the cost of asking.
Pairing on the first use	When a team adopts a pattern for the first time, an architect works alongside them through it. The second time, they do it alone. This is what makes adoption stick.
Worked examples	A complete, running reference service for each pattern. Engineers learn far more from reading working code than from reading prose about it.
Explaining the reasoning	Every standard is published with the argument behind it, including what was considered and rejected. People follow rules they can defend to a colleague.
Design brief coaching	Helping teams write a clear brief before their first review, so their first experience of the process is a good one.
Growing local advocates	Identifying one engineer inside each team who knows the standards well. Guidance from a teammate carries far more weight than guidance from a central group.
Onboarding material	A single starting page for new joiners covering how services are built here, so knowledge does not depend on who happens to sit nearby.

What it looks like when this is missing. Standards are treated as bureaucracy. Teams do the minimum to pass the automated checks. The central group becomes a bottleneck because nobody else can make architectural decisions.

How you know it is working. Teams start answering each other's architecture questions without the central group present. That is the clearest possible signal that the knowledge has genuinely transferred.

4.8 Component 8 — Reusable Assets, the Paved Road

What it is. The standards turned into things you can install and run. A written standard requires effort to follow. An asset makes following it the path of least resistance. This component is what makes the difference between a documentation project and an engineering one.

The assets

Asset	What it gives a team
Service templates	A command that generates a complete, working, already-compliant service skeleton: logging, tracing, authentication, health checks, timeouts, a pipeline and a dashboard, all present before a single line of business logic is written.
Shared libraries and SDKs	Common behaviour packaged as a dependency. Correlation identifiers, retries, structured logging and metrics come from the library, so a fix or improvement reaches every service through a version bump.
Infrastructure modules	Networks, clusters, databases and queues defined once as reusable code. A team requests a database rather than assembling one, and it arrives correctly configured, backed up and monitored.
CI/CD pipeline templates	The whole deployment sequence as a template a team includes rather than copies. Adding a new mandatory security scan is one central change, not six.
Dashboards and alerts as code	Monitoring generated from the same definitions, so it exists from day one and stays consistent.
Policy-as-code checks	The standards expressed as automated rules that run on every change, so the rule is enforced by a machine at the moment of the mistake rather than by a person weeks later.

Why this component decides whether the project succeeds

Standards that exist only as documents decay quietly. People mean to follow them, then a deadline arrives. Turning each standard into a template, a library or an automated check means teams get it by default and would have to work harder to avoid it.

The rule of thumb applied throughout: if a standard cannot be expressed as something installable or automatically checkable, it is probably too vague to be a standard yet.

4.9 Component 9 — Decision Records and Waivers

What it is. Two small pieces of paperwork that keep the whole system honest. A decision record captures what was decided and why. A waiver is written, time-limited permission to do something differently from the standard.

Architecture decision records

A decision record is a single page written at the moment a significant choice is made. It contains the context, the options considered, the decision, and the consequences accepted. Its value is almost entirely felt later: eighteen months on, when someone asks why the system works this way, the answer exists in writing instead of having left the company. It also prevents the same debate being re-run every year.

Waivers and exceptions

A waiver records that a team may deviate from a standard, why, and for how long. Waivers matter more than they look. A standards system with no exception route is one that people quietly route around, and once that starts you lose visibility of everything. A visible, expiring exception keeps the deviation known, owned and revisited.

Property of a waiver	Why it matters
Has a named owner	Someone is accountable for revisiting it.
Has an expiry date	Prevents a temporary exception becoming permanent by silence.
States the reason	Distinguishes a considered trade-off from an oversight.
States the added risk	Makes the cost visible to the people accepting it.
Is visible to everyone	Repeated waivers for the same rule are the clearest evidence that the rule itself is wrong.

> **The waiver register is a feedback instrument**
>
> If five teams request a waiver for the same rule, the rule is the problem, not the teams. Tracking waivers centrally turns individual friction into an obvious signal about which standard needs to change.

5. The Architecture, Layer by Layer

The diagram on the next page shows the whole foundation on one page. Read it from the top down: the teams at the top are the people who use the system, and each layer beneath them exists to serve the layer above it. The arrow running up the right-hand side is the feedback loop, and it is the part that keeps the whole thing alive rather than slowly going stale.

5.1 How to read the diagram

- Each dashed band is one layer. Layers are stacked by how abstract they are: decisions at the top-middle, working software towards the bottom.
- Downward arrows mean “turns into” or “is served by”. A standard turns into a template; a template deploys onto the platform.
- The italic text between layers explains what the arrow actually represents, so no arrow is left to interpretation.
- The red arrow on the right is the only upward path. It represents lessons from production changing the standards above.

Figure 2 | End-to-end architecture of the shared foundation

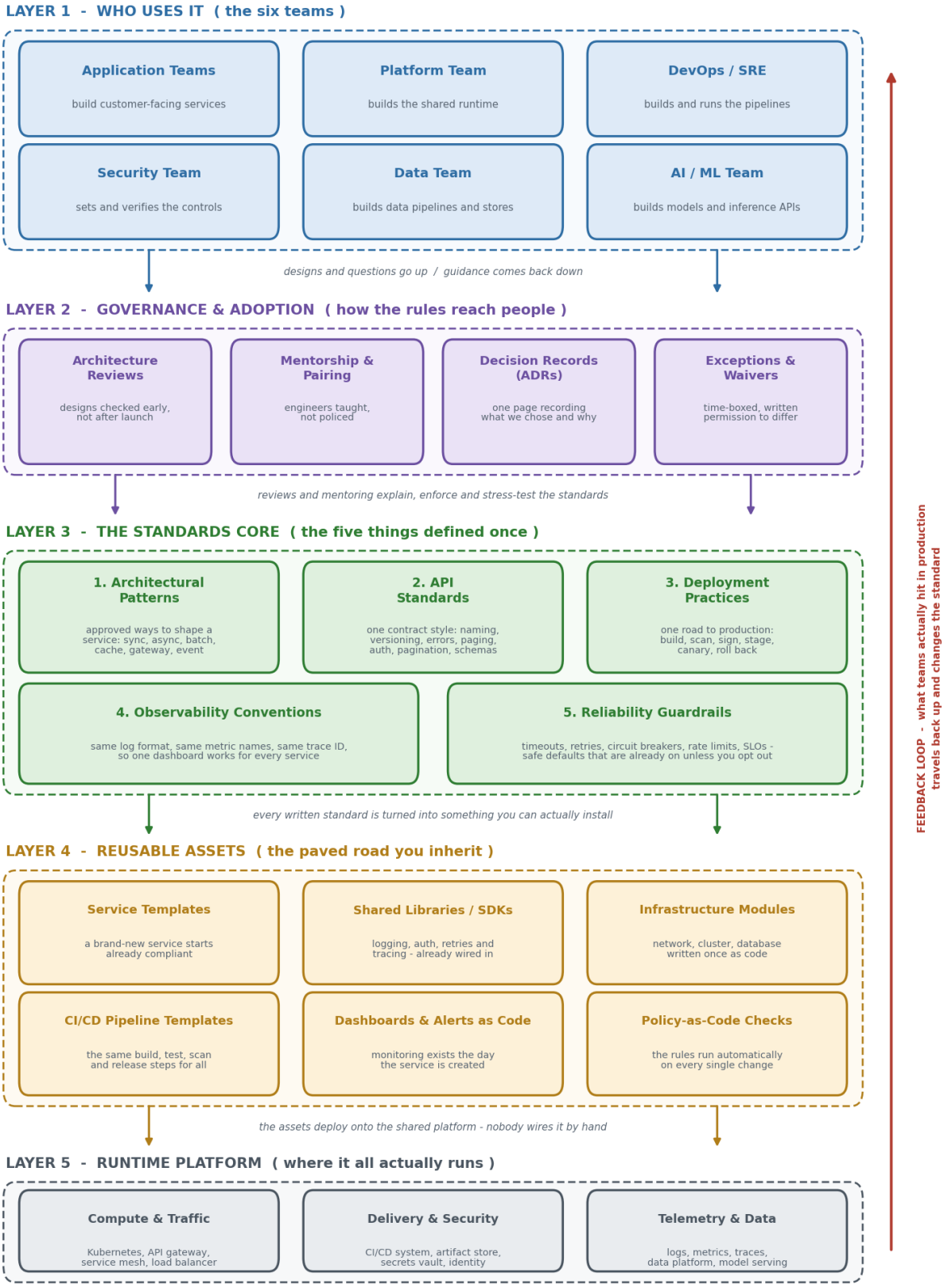

Figure 2: the complete architecture. Five layers, from the teams who use the foundation down to the platform it runs on, with the feedback loop that keeps the standards current.

5.2 Layer 1 — Who uses it: the six teams

This is the top layer because everything below exists to serve it. If a layer below stops being useful to these teams, it should be changed or deleted rather than defended.

What matters architecturally is that all six teams are treated as first-class consumers of the same foundation. It would have been easier to build something that suited application teams and let the data and AI teams work around it. That shortcut is exactly what creates the next generation of duplication, so it was avoided deliberately. Where a genuine difference existed, for example that inference calls are far slower than ordinary API calls, the standard was widened to cover the case rather than the team being excluded from it.

The two-way relationship with the layer below is the important part. Designs and questions travel down into the governance layer; guidance, decisions and templates travel back up.

5.3 Layer 2 — Governance and adoption: how the rules reach people

This layer is the human interface to the standards. It exists because a standard that nobody encounters at the right moment might as well not exist. It has four parts, each catching a different situation.

Part	What it catches
Architecture reviews	Problems in a design, before anyone has spent weeks building it. Also the only place cross-team duplication is visible.
Mentorship and pairing	Misunderstandings. It converts “I followed the rule” into “I understand what the rule protects me from”.
Decision records	Lost context. Prevents the same decision being re-litigated every year by people who never saw the original reasoning.
Exceptions and waivers	Legitimate mismatch. Provides a visible, expiring route to deviate, so deviation stays known instead of going underground.

This layer sits between the teams and the standards on purpose. Standards do not enforce themselves and they do not explain themselves. Without this layer, the layers below become a well-written wiki that nobody reads.

5.4 Layer 3 — The standards core: the five things defined once

This is the centre of the whole project: the five components described in sections 4.1 to 4.5. They sit at the middle of the stack because they are the source of everything below and the subject of everything above. Note that this layer contains no software at all. It is purely decisions, written down.

They are grouped as five rather than fifty deliberately. A large catalogue of rules is unusable, and unusable rules are ignored, which is worse than having none. Each of the five answers one clear question:

Standard	The question it answers
Architectural patterns	What shape should this thing be?
API standards	How does it talk to other things?

Standard	The question it answers
Deployment practices	How does it safely get to production?
Observability conventions	How do we tell what it is doing?
Reliability guardrails	How do we stop it taking everything else down?

Together these cover the entire life of a service: design, integration, release, operation and failure. That completeness is why five is enough.

5.5 Layer 4 — Reusable assets: the paved road

This layer is where written decisions become working software. Every item here traces directly back to a standard in the layer above, and that traceability is a design rule: an asset with no parent standard is unexplained magic, and a standard with no child asset is unenforced advice.

Standard above	Becomes this asset below
Architectural patterns	Service templates and reference implementations
API standards	Contract linters and generated client libraries
Deployment practices	CI/CD pipeline templates and policy-as-code checks
Observability conventions	Shared logging and tracing libraries, dashboards as code
Reliability guardrails	Shared resilience library with safe defaults pre-set

The phrase “paved road” captures the intent. Teams are not forced down it. It is simply so much smoother than the alternative that leaving it becomes a deliberate choice requiring a reason, which is precisely what the waiver process is for.

5.6 Layer 5 — Runtime platform: where it all actually runs

The bottom layer is the real infrastructure. It is at the bottom because everything above eventually lands here, and because it should be the least visible layer to an application engineer. If a team is thinking hard about this layer, an abstraction above it has failed.

Group	What it contains and what it does
Compute and traffic	The cluster that runs the containers, the gateway that is the front door for external callers, the service mesh that handles service-to-service communication, and the load balancer that spreads traffic.
Delivery and security	The system that runs the pipelines, the registry that stores signed build artifacts, the vault that holds credentials so they are never in code, and the identity system that decides who may do what.

Group	What it contains and what it does
Telemetry and data	Where logs, metrics and traces are collected and queried, plus the data platform and the model-serving infrastructure the data and AI teams depend on.

5.7 The feedback loop

The upward arrow is the most important line in the diagram, and the one most often missing from real architecture programmes. Without it, standards are written once by people who were correct at the time and gradually become wrong as reality moves on. With it, the foundation improves every time something goes wrong.

Four things travel up the loop:

- **Incidents.** Every post-incident review asks one extra question: was a standard wrong, missing, or right but unenforced? The answer becomes a change to the layer above.
- **Waivers.** Repeated requests for the same exception are treated as evidence that the standard needs to change, not that the teams need more discipline.
- **Review friction.** If the same argument keeps happening in reviews, the standard is unclear and needs rewriting.
- **Adoption data.** A template nobody uses is a template that does not fit. Low adoption is a design defect in the asset, not a failure of the team.

5.8 Why the layers are ordered this way

Design principle	What it means and why it was chosen
Decisions above implementations	The standards layer sits above the assets layer, so an asset can be rebuilt on new technology without changing the decision it implements. This is what stops the foundation being permanently tied to today's tooling.
Every rule has an owner	Each standard, asset and pattern has a named owner. Unowned rules rot, and rotten rules teach people to ignore all rules.
Enforcement close to the mistake	Checks run in the pipeline, seconds after a change, not in a quarterly audit. A fast, boring correction is cheap; a slow one is an argument.
An explicit escape hatch	The waiver route exists at the governance layer by design, because a system with no legitimate exit is one people leave illegitimately.
Teams stay autonomous	The foundation standardises how things are built, never what is built. Teams keep full ownership of their product decisions.

6. The Workflow, End to End

This section follows a single piece of work from the moment someone says “we need a new service” to the moment it is running in production and improving the standards that created it. The diagram summarises the ten stages; the text after it explains each one in detail.

Figure 3 | End-to-end workflow, from idea to running service

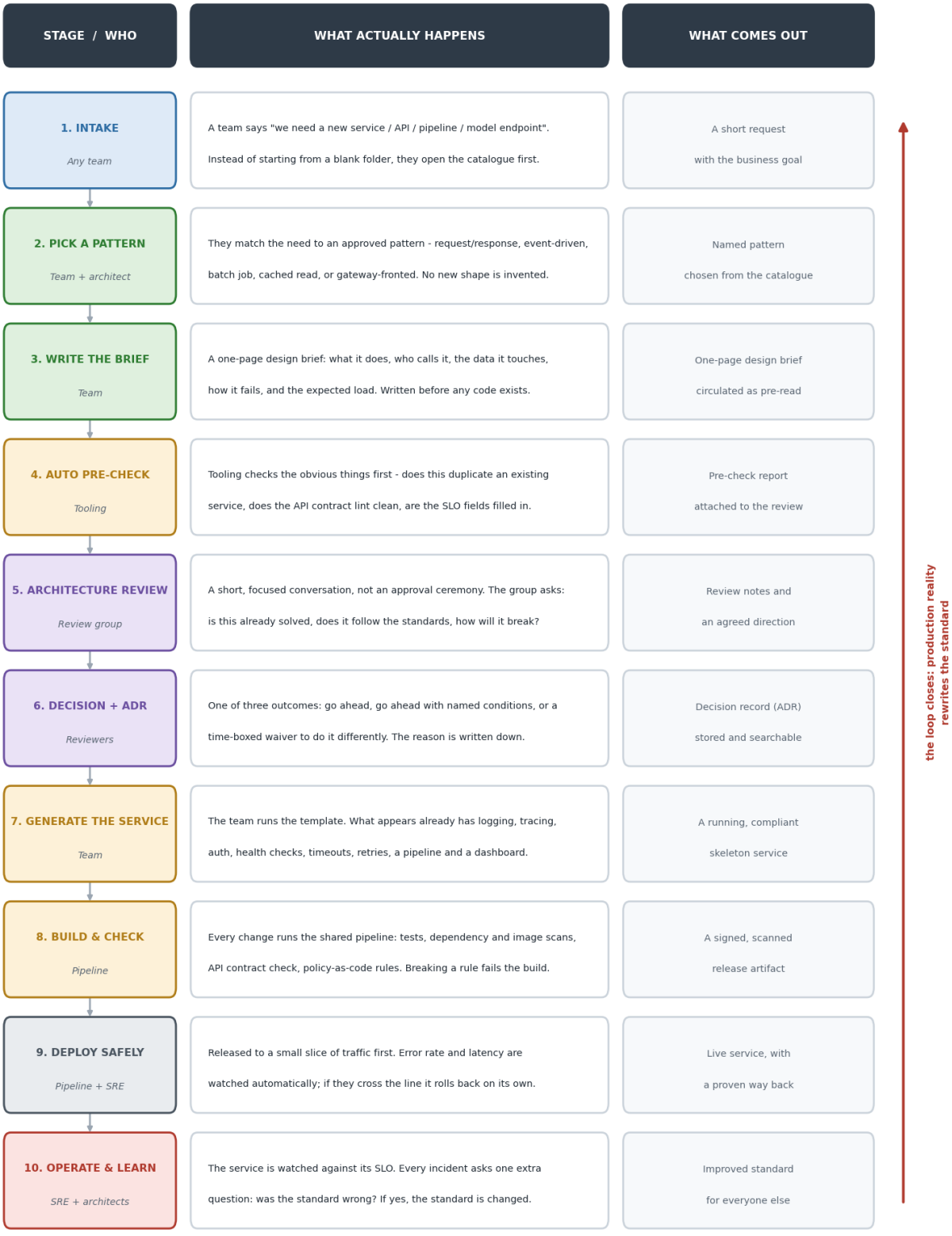

Figure 3: the ten stages, showing who is involved, what actually happens, and what each stage produces.

6.1 Stage 1 — Intake

Who. Any of the six teams.

What happens. A team identifies a need: a new service, a new API, a new data pipeline, a new model endpoint. Under the old way of working, the next step was to create an empty repository. Under the new way, the first step is to check the catalogue and the service inventory to see whether the capability already exists somewhere.

Why this stage exists. The cheapest service to build is the one you discover you do not need to build. This single habit is responsible for a large share of the duplication that was eliminated.

What it produces. A short statement of the business need, and a preliminary answer to whether anything similar already exists.

6.2 Stage 2 — Choose a pattern

Who. The team, usually with a short conversation with an architect.

What happens. The need is matched to an entry in the pattern catalogue. Does the caller need an immediate answer, or can the work happen in the background? Is the data read far more often than it changes? Is this a continuous flow of records? Each answer points at a specific pattern.

Why this stage exists. Choosing a shape from a short list takes minutes. Inventing one takes days and produces something only its author fully understands.

What it produces. A named pattern, recorded so that everyone downstream knows what to expect.

6.3 Stage 3 — Write the one-page design brief

Who. The team.

What happens. The engineer fills in the standard brief described in section 4.6: what it does, who calls it, which pattern, what data it touches, how it fails, expected load, and who operates it.

Why this stage exists. Writing forces clarity. A significant number of designs are corrected by the author while filling in the “how it fails” field, before anyone else has even read it. It is deliberately one page, because the goal is thinking, not documentation.

What it produces. A brief circulated as pre-read, so the review meeting can start with everyone informed.

6.4 Stage 4 — Automated pre-check

Who. Tooling. No humans involved.

What happens. Automated checks run before any person spends time on the design. Does a service with a similar name or purpose already exist in the inventory? Does the draft API specification pass the contract linter? Are the required fields, including the SLO target and the owner, actually filled in?

Why this stage exists. Human review time is the scarcest resource in the whole workflow. Spending it on things a machine can check is waste. Reviewers should only ever see problems that need judgement.

What it produces. A pre-check report attached to the review, so the meeting starts from a known baseline.

6.5 Stage 5 — Architecture review

Who. The proposing engineer, an architect, and relevant specialists.

What happens. A thirty-minute conversation structured around four questions: does this already exist, does it follow the standards, how will it fail, and who will operate it. The tone is advisory. The reviewer's job is to improve the design and surface reuse, not to grant permission.

Why this stage exists. This is the only point in the entire process where someone with a view across all six teams looks at the design. Remove it and cross-team duplication becomes invisible again.

What it produces. Review notes and an agreed direction.

6.6 Stage 6 — Decision and record

Who. The reviewers, with the team.

What happens. One of three outcomes is recorded. Proceed. Proceed with named conditions, each tracked as a real work item. Or a time-boxed waiver permitting a documented departure from the standard. The reasoning is written into a decision record.

Why this stage exists. An undocumented decision is a decision that will be re-argued. The record is also what makes the waiver register useful as a feedback signal about which standards are wrong.

What it produces. A stored, searchable decision record.

6.7 Stage 7 — Generate the service

Who. The team.

What happens. The team runs the template for their chosen pattern. What appears is not an empty folder. It is a running service that already has structured logging, trace propagation, authentication, health checks, sensible timeouts and retries, resource limits, a working CI/CD pipeline, a dashboard and a set of default alerts.

Why this stage exists. This is where the entire project pays off. Everything the standards describe is delivered in one command, so following the standard costs the team nothing and skipping it would cost them days.

What it produces. A compliant, running skeleton service, ready for business logic.

6.8 Stage 8 — Build and check

Who. The shared pipeline.

What happens. Every change runs the same sequence: tests, dependency and image scanning, secret detection, API contract validation, and the policy-as-code rules. Breaking a rule fails the build, with a message explaining which rule and why it exists.

Why this stage exists. Enforcement has to happen at the moment of the mistake to be cheap. A rule caught in seconds is a correction; the same rule caught in an audit three months later is a project.

What it produces. A signed, scanned, recorded release artifact.

6.9 Stage 9 — Deploy safely

Who. The pipeline, with SRE oversight.

What happens. The artifact goes to staging and passes smoke tests, then to a small slice of production traffic. Error rate and latency are compared automatically against the current version. If the new version looks worse, traffic returns to the old one without waiting for a human decision. If it looks fine, the slice widens gradually to everything.

Why this stage exists. It changes the question from “will this release break production?” to “how quickly will we notice and undo it?”. The second question has a much better answer.

What it produces. A live service, with a proven route back.

6.10 Stage 10 — Operate and feed back

Who. SRE and architects, with the owning team.

What happens. The service runs against its declared SLO on dashboards that already existed before launch. When something goes wrong, the post-incident review asks the usual questions and one extra one: was a standard wrong, missing, or right but not enforced? The answer changes the foundation.

Why this stage exists. This closes the loop. Without it the standards are a snapshot of what was true when they were written, and they get less accurate every month.

What it produces. An improved standard, template or guardrail that every other team inherits automatically.

6.11 A worked example: a new Refunds API

To make the workflow concrete, here is the same ten stages applied to one realistic piece of work.

Stage	What happened
1. Intake	The application team needs customers to request a refund. A search of the inventory shows a payments service exists but has no refund capability, so this is genuinely new.
2. Pattern	The customer waits for confirmation, so request-response is chosen. The actual money movement is slow and involves an external provider, so an event is published for that part. Two patterns, combined deliberately.
3. Brief	Writing the “how it fails” field surfaces the important question: what happens if the payment provider is unreachable after the customer has been told the refund is accepted?
4. Pre-check	The contract linter flags that the error format is non-standard and that the endpoint is named as an action rather than a resource. Both are fixed before any human reads it.
5. Review	Security asks how refund permissions are checked. Operations asks what happens to duplicate submissions. The idempotency rule from the API standard answers the second question directly.
6. Decision	Proceed, with two conditions: an idempotency key on the submission endpoint, and a documented reconciliation process for refunds accepted but not completed. Both recorded.
7. Generate	The template produces the service. Logging, tracing, authentication, health checks, retries, pipeline and dashboard are present before the first line of refund logic is written.
8. Build	The pipeline blocks the first attempt because the new dependency has a known vulnerability. The team upgrades it the same afternoon.
9. Deploy	Released to two percent of traffic. Latency is normal, error rate is flat, so the slice widens to full traffic over the following hour.
10. Feed back	Two weeks later an incident shows the provider timeout was set too high, causing requests to queue. The default in the shared library is lowered, and every service using it inherits the fix on its next release.

6.12 What happens when a team disagrees

Disagreement is expected and is handled explicitly, because a process with no legitimate way to say no produces quiet non-compliance instead of honest debate.

1. The team states the mismatch: which standard does not fit, and what it costs them to follow it.
2. The standard is examined first, not the team. If the objection is sound, the standard changes, and the change benefits everyone.
3. If the standard is right but this case is genuinely unusual, a time-boxed waiver is granted with a named owner, an expiry date and a written statement of the added risk.
4. The waiver is recorded in the register. If the same waiver is requested by several teams, that is treated as proof the standard is wrong.

7. Driving Adoption

Defining standards is the smaller half of the work. Getting six busy teams to actually use them is the harder half, and it is where most initiatives of this kind fail. Adoption was treated as five distinct rungs, each one making the standard harder to ignore than the last.

Figure 4 | How adoption was driven - each rung makes the standard harder to ignore

Figure 4: the five rungs of adoption, from writing a standard down to measuring whether it is used.

Rung	What it involves and why it is necessary
1. Write it down	One page, one place, with the reasoning included. A standard that lives in someone's head or in a meeting recording is not a standard. This rung is necessary but by itself achieves almost nothing.
2. Make it the default	Ship it as a template, a library or a pipeline. This is the largest single jump in adoption, because it changes the standard from something a team must do into something they receive.
3. Teach it	Reviews, pairing, office hours and worked examples. People follow rules they understand and quietly work around rules they do not. This rung converts compliance into judgement.
4. Check it automatically	Policy-as-code in the pipeline. Enforcement by machine at the moment of the mistake is impersonal, immediate and cheap. Enforcement by person, weeks later, is an argument.
5. Measure it	Adoption percentage, open waivers, incidents traced to a missing standard. Without numbers you cannot tell the difference between a standard that is working and one that is being politely ignored.

7.1 How resistance was handled

Objection heard	How it was addressed
“This will slow us down.”	Measured against reality: generating a compliant service takes minutes, while hand-building the same plumbing takes days. Where following the standard genuinely was slower, that was treated as a defect in the asset and fixed.

Objection heard	How it was addressed
“Our team is different.”	Often true, particularly for data and AI work. The response was to widen the standard to cover the case, not to exclude the team. Exclusion is how the next generation of duplication begins.
“We already have our own version.”	Existing work was not forcibly rewritten. New work used the standard, and migration happened only where there was a clear benefit.
“The rule does not make sense here.”	Treated as a genuine signal. Either the standard changed, or a time-boxed waiver was granted with the reasoning recorded.

8. How Success Was Measured

A foundation project that cannot show its effect will eventually be defunded, so measurement was built in from the start. The measures below are deliberately split into two groups: what is used, and what improved as a result.

8.1 Adoption measures

Measure	What it tells you
Percentage of new services created from a template	Whether the paved road is genuinely the easiest path, or just the recommended one.
Percentage of services emitting standard telemetry	Whether the observability conventions are actually in force everywhere.
Number of open waivers, by standard	Which standards are causing friction and probably need rewriting.
Reviews completed, and average turnaround time	Whether governance is helping or has become a bottleneck.
Number of distinct pipeline definitions in use	A direct measure of remaining duplication. It should fall towards one.

8.2 Outcome measures

Measure	What improvement looks like
Time from idea to first running service	Falls sharply, because the plumbing is inherited rather than built.
Deployment frequency	Rises, because releasing stops being frightening.
Change failure rate	Falls, because every change passes the same checks.
Time to restore service after an incident	Falls, because telemetry is consistent and rollback is automatic.
Number of services affected per incident	Falls, because guardrails contain failures instead of letting them spread.
Onboarding time for a new engineer	Falls, because every service is recognisably built the same way.
Effort to produce compliance evidence	Falls, because the pipeline records it as a by-product of normal work.

9. Risks and How They Were Managed

Risk	Why it matters and how it was handled
The standards become a bottleneck	If every change needs central approval, teams route around the process. Handled by keeping reviews advisory by default, restricting them to genuinely new or cross-cutting work, and enforcing routine rules by automation instead of by meeting.
The foundation goes stale	Standards written once slowly become wrong. Handled by the feedback loop: incidents, waivers and adoption data all feed changes back into the standards, and every standard has a named owner and a review date.
Standardising the wrong things	Standardising business logic destroys team autonomy and creates resentment. Handled by scoping the foundation strictly to how things are built, never to what is built.
Rules without understanding	Teams satisfy the automated check and still produce something fragile. Handled by the mentorship component, and by publishing the reasoning behind every standard rather than just the rule.
One team's needs dominate	A foundation shaped only around application services fails the data and AI teams. Handled by including all six teams in the design and widening standards rather than granting blanket exclusions.
Exceptions become permanent	A temporary waiver silently becomes the norm. Handled by requiring every waiver to have an owner, an expiry date and a stated risk, and by reviewing the register regularly.

10. Glossary

Every term used in this document, explained without jargon.

Term	Plain-English meaning
API	The doorway a service exposes so other software can use it. A menu of things you can ask it to do.
Architectural pattern	A proven, reusable way of arranging the parts of a service, along with when to use it and when not to.
ADR (Architecture Decision Record)	A one-page note recording an important decision, the options considered, and the consequences accepted.
Artifact	The packaged output of a build. The exact thing that gets deployed.
Bulkhead	Separate resource pools per dependency, so one slow dependency cannot starve everything else.
Canary release	Sending a new version to a small slice of real traffic first, to see whether it behaves before everyone gets it.
Circuit breaker	After repeated failures, stop calling a struggling service for a while so it can recover instead of being hammered.
CI/CD	The automated sequence that builds, tests, checks and releases code.
Contract (API contract)	The written, machine-readable promise about how an API behaves, which callers depend on.
Correlation ID	One identifier attached to a request and carried through every service it touches, so its full journey can be traced.
Guardrail	A safe default, already switched on, that limits how badly something can go wrong.
Idempotency	The property that doing the same operation twice has the same effect as doing it once. Prevents duplicate charges when a network retries.
Latency	How long something takes to respond.
Observability	Being able to tell what a system is doing from the outside, using logs, metrics and traces.
Paved road	The supported, easy default path. Teams may leave it, but the smooth route is deliberately the compliant one.
Policy-as-code	Rules written as automated checks that run in the pipeline, rather than as prose in a document.
Rate limit	A ceiling on how many requests a caller may make in a period.
Rollback	Returning to the previous known-good version.

Term	Plain-English meaning
Service mesh	Infrastructure that handles service-to-service communication, including retries, encryption and traffic routing.
SLO (Service Level Objective)	A written target for how reliable a service should be, for example 99.9 percent of requests succeeding.
Error budget	The agreed allowance for falling short of an SLO. When it is used up, reliability work takes priority over features.
Template / scaffold	A generator that produces a complete, already-compliant starting service.
Timeout	The maximum time to wait for an answer before giving up.
Trace	The full record of one request's journey across services, showing where the time went.
Waiver	Written, time-limited permission to depart from a standard, with the reason and added risk recorded.

11. Appendix: Mapping Back to the Original Description

For anyone who needs to connect this document back to the one-sentence version of the project, here is the mapping.

Part of the original sentence	Where it is explained in this document
Eliminated duplicated infrastructure work	Sections 2 and 8.2, with Figure 1
Across application, platform, DevOps, security, data and AI teams	Section 3.3 and Layer 1 in Section 5.2
Reusable architectural patterns	Section 4.1
API standards	Section 4.2
Deployment practices	Section 4.3
Observability conventions	Section 4.4
Reliability guardrails	Section 4.5
Driving adoption	Section 7, with Figure 4
Architecture reviews	Sections 4.6 and 6.5
Mentorship	Section 4.7

The single sentence summary, in plain English

Six teams were each rebuilding the same foundations. This project defined one shared set of proven building blocks, turned them into templates and automated checks so teams received them by default, and used reviews and mentoring to make sure they were understood and actually used. The result is that engineers spend their time on the things that are different about their work, rather than rebuilding the things that are the same.